

PROG2002 – Web Development II
Assignment 2: Use Case (A Dynamic Website)

Student ID: _____ **Last Name:** _____
First Name: _____

Title of the project: Charity Events Sydney – A Dynamic Charity Event
Management Website

Date submitted: _____

GitHub repository: <https://github.com/>_____/charity-events-a2
(private repository; the marker is invited as a collaborator)

Demo video (SCU OneDrive link): _____

Introduction/Motivation

This report documents the analysis, design and development of **Charity Events Sydney**, a dynamic client-side website that allows the public to discover and register for charity events hosted by charitable organisations in their city. The project was developed for Assessment 2 of PROG2002 Web Development II and delivers three integrated layers: a MySQL database, a RESTful API built with Node.js and ExpressJS, and a browser-based website built with HTML, CSS, JavaScript, the DOM and Promises.

A charity event is an organised activity – such as a gala dinner, fun run, silent auction or concert – hosted by a charitable organisation to raise funds and awareness for a specific cause. The brief for this assessment asks for a platform that connects those organisations with potential attendees and handles the event-viewing and registration journey. That framing shaped every design decision in this project: the website is not a generic event listing, it is a **donation-focused** platform where every event publishes the cause it supports and the financial goal it is trying to reach.

Project scope

Assessment 2 covers the read-only, public-facing half of the platform. The following table separates what this submission delivers from what is deliberately deferred to Assessment 3.

Layer	Delivered in Assessment 2	Deferred to Assessment 3
Database	Events, categories and charitable organisations; primary and foreign keys; 14 sample events across 5 categories	Users, registrations and ticket purchases; payments; audit tables
API	GET endpoints for the Home, Search and Event Detail pages, plus category lists (read-only)	POST, PUT and DELETE endpoints to create and manage events and to process registrations
Website	Home, Search events and Event detail pages with live API data, filtering, validation and DOM-rendered feedback	Ticket purchasing, user accounts, attendee dashboard and an admin side for organisations

Motivation

Small charitable organisations rarely have the budget for a commercial event platform, and the platforms that do exist are usually built for ticketed entertainment rather than for fundraising transparency. The motivation for this project is therefore twofold. First, from a technical standpoint, the case study is a realistic exercise in client–server communication: it forces a genuine separation between the data layer, the API that exposes it and the browser code that consumes it. Second, from a domain standpoint, the design goal was to make **fundraising progress visible**, because donors give more readily when they can see exactly what an event is raising money for and how close it is to the goal. That single idea – publish the goal, show the progress – runs through the data schema, the API response shape and the user interface.

Terminology note

The assessment criteria describe the scenario as a "crowdfunding" or "fundraiser" platform, while the task description uses "charity events". In this project those terms describe the same thing: the crowdfunding element is implemented as the **donation component of an event ticket**, and the "fundraiser data" referred to in the rubric is the event, organisation, category and fundraising-goal data described in the Data Schema section below.

Problem Statement

Charitable organisations in a large city face three connected problems when they try to fill their fundraising events. Each one is addressed by a specific part of this project.

Problem 1 – Event information is scattered and hard to compare

A supporter who wants to support a local cause has to visit each organisation's own website or social media page to find out what is happening, when, where and at what price. There is no single place to see every upcoming charity event, and event pages rarely state what the money being raised is actually for. As a result, potential attendees either miss events that would interest them or lose confidence that their money will be used as intended.

How the website addresses it: a Home page lists all active and upcoming events in one place, each card showing the event name, category, date, venue, ticket price and a fundraising progress bar. The listing is retrieved live from the API rather than hard-coded.

Problem 2 – Supporters cannot search by what matters to them

Even when event information is published, it is usually presented as a chronological blog feed. Someone who is free only on a particular weekend, lives in a particular suburb, or prefers a particular type of activity (for example a fun run rather than a formal dinner) has to read every listing to find a match. Free-text search alone does not solve this, because the criteria supporters care about are structured: a date, a place and a category.

How the website addresses it: a dedicated Search events page provides a filtering form with exactly those three criteria – date, location and category – which can be used individually or combined. The form calls a dedicated search endpoint on the API and renders the matching events immediately, with clear validation and empty-state feedback.

Problem 3 – Organisations cannot enforce fundraising policy, and donors cannot judge credibility

Organisations occasionally run promotions that breach their own fundraising policy or state licensing requirements. Once such an event is advertised it is difficult to withdraw, and publishing it damages the organisation's credibility. At the same time, donors have no easy way to see whether an event is legitimate or how much of its goal has already been met.

How the website addresses it: the database records an event status. Events marked as **suspended** for breaching policy are excluded by the API and therefore never appear on the Home page, in search results or on a detail page – even if a user navigates directly to the event ID, the API responds with 404. Every visible event publishes its purpose statement, its goal amount and the amount raised so far.

Summary of the problem space

ID	Problem	Addressed by	Assessment part
P1	Event information is scattered; supporters cannot see what is being raised	Home page dynamic event listing with goal progress	Parts 1–3
P2	No structured way to find events by date, location or category	Search events page with a three-criteria filter form	Parts 2–3
P3	Policy-breaching events cannot be withheld; donors cannot assess credibility	Event status field enforced by the API; published goals and progress	Parts 1–2

Solution

The solution is a three-tier web application. It follows the client–server model that underpins this unit: the browser is the client, the Node.js/Express process is the server, and MySQL is the persistent data store. No data is baked into the HTML pages; the browser requests it from the API, the API queries the database, and the response is rendered into the page with JavaScript and the DOM.

Architecture

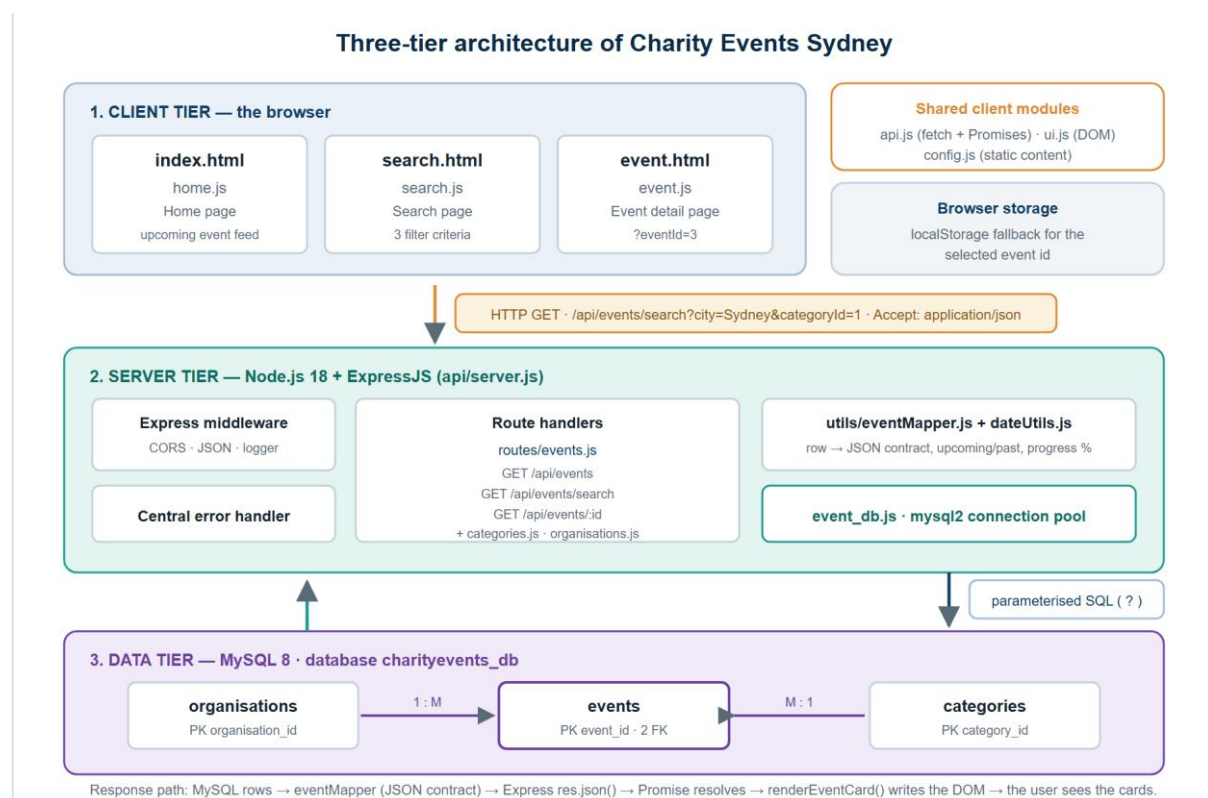


Figure 1 — Three-tier architecture: browser client, Node.js/Express server, MySQL data tier, and the request/response path between them.

Client–server communication in detail

The diagram above shows the static structure. The numbered sequence below describes what actually happens when a user interacts with the site, using the Home page as the example.

- **Step 1 – Page load.** The browser requests index.html from the Express static middleware. The page is delivered as plain HTML and CSS with no event data embedded in it.

- **Step 2 – Script execution.** The browser loads config.js (hard-coded organisation text), api.js (the fetch wrapper), ui.js (DOM rendering helpers) and home.js (page logic) in that order.
- **Step 3 – API request.** home.js calls API.getEvents({ status: "upcoming" }), which issues GET /api/events with the Fetch API and returns a Promise.
- **Step 4 – Server processing.** Express routes the request to routes/events.js. The handler builds a parameterised SQL statement, and event_db.js executes it on a pooled mysql2 connection.
- **Step 5 – Database query.** MySQL joins events to categories and organisations so one row already contains everything the card needs, and excludes suspended events with a status filter.
- **Step 6 – Response shaping.** utils/eventMapper.js converts each database row into the JSON contract the front end expects, including calculated fields such as displayStatus ("upcoming" or "past"), the number of days until the event and the percentage of the fundraising goal reached.
- **Step 7 – JSON response.** Express serialises the result as { success, count, filter, data: [...] } and returns HTTP 200.
- **Step 8 – DOM rendering.** The awaited Promise resolves in home.js, which calls renderEventCard() for each event and writes the generated markup into the events container. Any error is caught and rendered into the message area instead.

Two design decisions are worth highlighting because they are the reason the site stays consistent. First, **all calculation that both the server and the client could perform is done on the server** (upcoming versus past, progress percentage, formatted dates and prices). This guarantees that the Home, Search and Detail pages always agree with one another and keeps the browser code focused on rendering. Second, **every endpoint returns the same event shape**, so a single renderEventCard() function serves all three pages.

Mapping of requirements to implementation

Requirement in the brief	Implemented by	Where to find it
Home page shows the organisation's general information	Hard-coded static content in the Home page header, mission and contact sections	client/index.html

Requirement in the brief	Implemented by	Where to find it
Home page lists current and upcoming events using an API	GET /api/events called by home.js on page load	client/js/home.js
Events marked "past" or "upcoming" from the event date	classifyEvent() running on the server; exposed as displayStatus	api/utls/dateUtils.js
Suspended events must not be shown	status column in the events table; API filters to active/cancelled only	api/routes/events.js
Search page form with date, location and category	Filter form with date pickers, a datalist-backed text field and a drop-down	client/search.html
One or several criteria can be combined	GET /api/events/search builds its WHERE clause from whichever parameters are present	api/routes/events.js
"Clear Filters" button that resets fields (DOM manipulation)	clearFilters() resets every field and re-renders the list	client/js/search.js
Error messages shown with basic DOM manipulation	showMessage() builds and inserts a message element	client/js/ui.js
Event detail page for the selected event only	Event ID passed in the query string, with localStorage as a fallback	client/js/event.js
Full event description, ticket information and goal vs progress	GET /api/events/:id returning description, purpose, ticket and goal objects	api/routes/events.js
"Register" button triggers a modal/alert stating the feature is under construction	Modal dialog with the required message	client/event.html
Navigation menu present on every page	Shared header markup and styling across all three pages	client/*.html

Web UX

The user experience goal was that a first-time visitor should be able to answer three questions without instruction: **What is this site? What events can I attend? How do I find the one that suits me?** The design work below was directed at those questions rather than at decoration.

Wireframing

Each page was wireframed before any HTML was written, so that content hierarchy was decided first and visual styling second. The wireframes below were the working design documents for the build; the live pages follow them closely, as the screenshots later in this report show.



Figure 2 — Wireframes of the Home, Search events and Event detail pages, showing the shared navigation, the card anatomy and the three-criteria filter panel.

The implemented pages

The wireframes above were realised in HTML and CSS without any UI framework. The screenshots below are captured from the running application served by api/server.js, with data loaded live from the API, so they show the real rendering rather than a mock-up.

Every ticket tells a story

Charity Events Sydney brings together the Harbour Lights Foundation, Green Coast Conservation Trust and Open Table Community Kitchen. Browse their upcoming galas, fun runs, auctions and concerts, then register to turn a night out into real change for families, coastlines and neighbours in our city.

Browse upcoming events

Search events

10

Upcoming events

5

Event categories

3

Charitable organisations

\$186,850

Raised towards goals

Our mission

We connect people who want to give with the causes that need them most. Every event listed on this website is hosted by a registered charitable organisation, and every ticket purchased is treated as a donation towards a specific, published goal.

Every goal is published

Each event shows exactly how much it is trying to raise and how far it has come, so donors can see where their money is going.

Local organisations only

All events are hosted by vetted charitable organisations working within our city and its surrounding regions.

Policy-checked listings

Events that do not meet our fundraising and licensing policy are suspended and are never shown on this website.

Upcoming charity events

Loaded live from the Charity Events REST API - GET /api/events

Filter these events

IN 5 DAYS



Community Market

Sunday Community Market

A free family market with produce stalls.

Thu, 1 Oct 2026 - 8:00 am

Petersham Town Hall Forecourt, Petersham

Community Drive **FREE**

Fundraising progress 29%

Free entry

View details →

IN 8 DAYS



Benefit Concert

Open Table Benefit Concert

An acoustic evening of live music.

Sun, 4 Oct 2026 - 7:00 pm

The Factory Theatre, Marrickville

Concert

Fundraising progress 80%

\$55.00

View details →

IN 12 DAYS



Coastal Walk

Green Coast Coastal Walk

A guided 12 km coastal walk with a clean-up.

Thu, 8 Oct 2026 - 7:00 am

Coogee Beach Surf Life Saving Club, Coogee

Fun Run **FREE**

Fundraising progress 45%

Free entry

View details →

IN ABOUT 3 WEEKS



Twilight Run

Harbour Lights Twilight Run 2025

A 5 km and 10 km sunset run along the harbour foreshore.

Sat, 17 Oct 2026 - 4:30 pm

Barangaroo Reserve, Barangaroo

Fun Run

Fundraising progress 47%

\$45.00

View details →

IN ABOUT 4 WEEKS



Silent Auction

Coastline Silent Auction

Bid on donated art, getaways and experiences.

Mon, 26 Oct 2026 - 6:00 pm

Bondi Pavilion Gallery, Bondi

Silent Auction

Fundraising progress 35%

\$35.00

View details →

IN ABOUT 5 WEEKS



Family Fun Day

Family Fun Day on the Green

Free entry, carnival games and face painting.

Sat, 31 Oct 2026 - 10:00 am

Camperdown Memorial Rest Park, Newtown

Community Drive **FREE**

Fundraising progress 16%

Free entry

View details →

IN ABOUT 6 WEEKS



IN ABOUT 9 WEEKS



IN ABOUT 10 WEEKS



Figure 6 — The Home page. Static organisation content sits above a dynamic listing of ten upcoming events returned by GET /api/events, each card carrying a status badge, category, venue, ticket price and a fundraising progress bar.



[Home](#) > [Search events](#)

Search charity events

Narrow down the events that match your interests. Choose one criterion or combine several — date, location and category all work together.

Filter events

All fields are optional. Leave a field blank to ignore that criterion.

1. DATE

From

To

yyyy/mm/日

yyyy/mm/日

Quick date options

☒ Anytime ☐ Next 30 days

☐ Next 90 days

Choosing a quick option fills the date fields for you.

2. LOCATION

City or suburb

e.g. Sydney, Bondi, Newtown

Partial matches work — "syd" finds Sydney.

3. EVENT CATEGORY

Category

All categories

5 categories available · loaded from GET /api/categories

4. TIME FRAME

☒ Upcoming events only

☐ Include past events

☐ Past events only

[Search events](#)

[Clear filters](#)

Upcoming events

10 events available — use the filters to narrow them down

IN 5 DAYS

Community Market

Sunday Community Market

A free family market with produce stalls.

Thu, 1 Oct 2026 · 8:00 am

Petersham Town Hall Forecourt, Petersham

Community Drive **FREE**

Fundraising progress

29%

Free entry [View details →](#)

IN 8 DAYS

Benefit Concert

Open Table Benefit Concert

An acoustic evening of live music.

Sun, 4 Oct 2026 · 7:00 pm

The Factory Theatre, Marrickville

Concert

Fundraising progress

80%

\$55.00 [View details →](#)

IN 12 DAYS

Coastal Walk

Green Coast Coastal Walk

A guided 12 km coastal walk with a clean-up.

Thu, 8 Oct 2026 · 7:00 am

Coogee Beach Surf Life Saving Club, Coogee

Fun Run **FREE**

Fundraising progress

45%

Free entry [View details →](#)

IN ABOUT 3 WEEKS

Twilight Run

Harbour Lights Twilight Run 2025

A 5 km and 10 km sunset run along the harbour foreshore.

Sat, 17 Oct 2026 · 4:30 pm

Barangaroo Reserve, Barangaroo

Fun Run

Fundraising progress

47%

\$45.00 [View details →](#)

IN ABOUT 4 WEEKS

Silent Auction

Coastline Silent Auction

Bid on donated art, getaways and experiences.

Mon, 26 Oct 2026 · 6:00 pm

Bondi Pavilion Gallery, Bondi

Silent Auction

Fundraising progress

35%

\$35.00 [View details →](#)

IN ABOUT 5 WEEKS

Family Fun Day

Family Fun Day on the Green

Free entry, carnival games and face painting.

Sat, 31 Oct 2026 · 10:00 am

Camperdown Memorial Rest Park, Newtown

Community Drive **FREE**

Fundraising progress

16%

Free entry [View details →](#)

IN ABOUT 6 WEEKS

Gala Dinner

Lights of Hope Gala Dinner

A black-tie dinner with a guest speaker and live auction.

[View details →](#)

IN ABOUT 9 WEEKS

Run for Your Supper

Run for Your Supper 10K

A chip-timed 10 km run through the inner west.

[View details →](#)

Figure 7 — The Search events page before any criteria are applied. The filter panel is sticky on desktop so the criteria remain visible while the user scrolls the results, and the category drop-down has been populated from GET /api/categories.

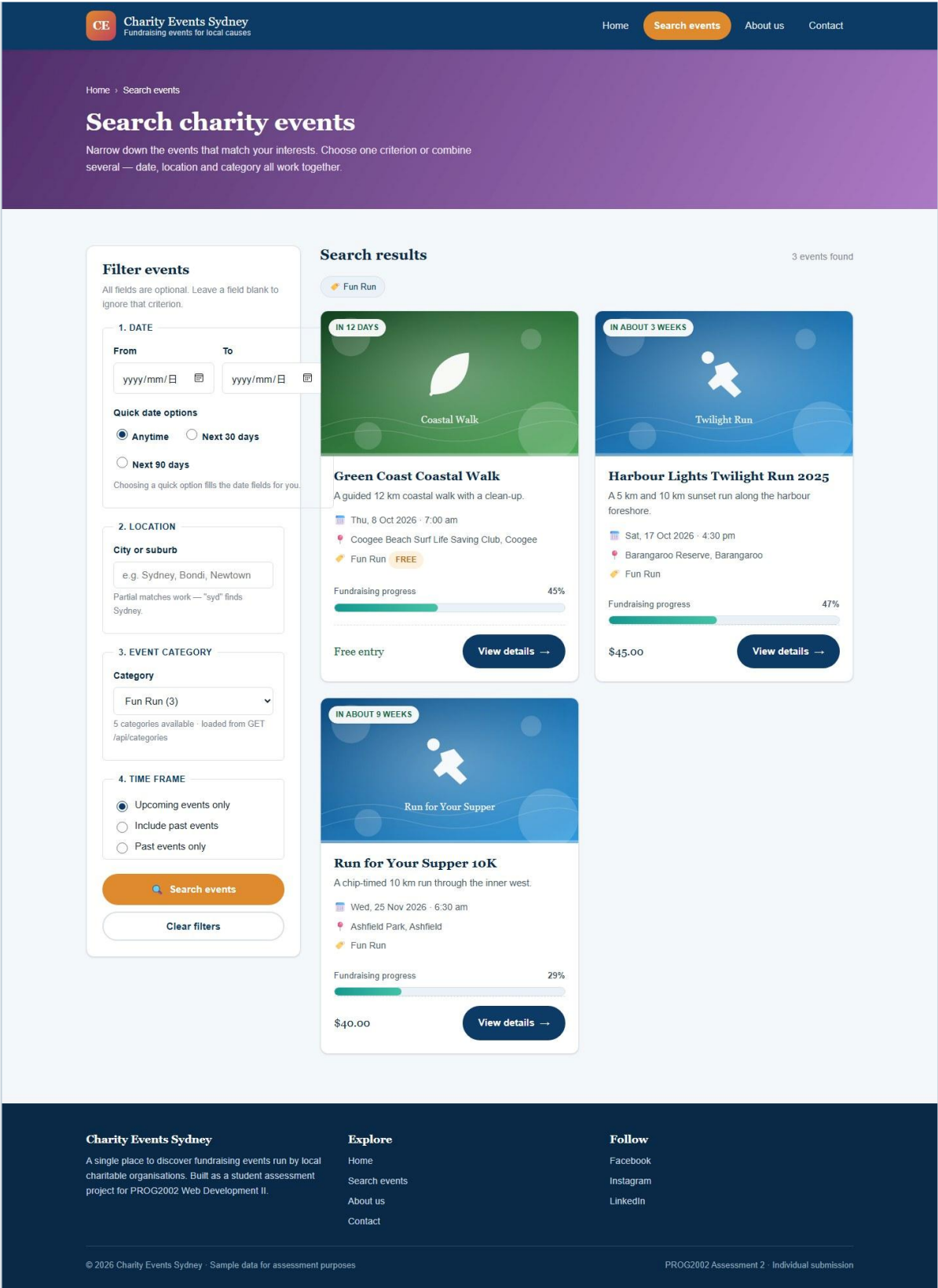


Figure 8 — The same page after filtering by category (Fun Run) with the time frame set to upcoming events. The query string, the active-filter chip, the result count and the result list all update from a single call to GET /api/events/search.

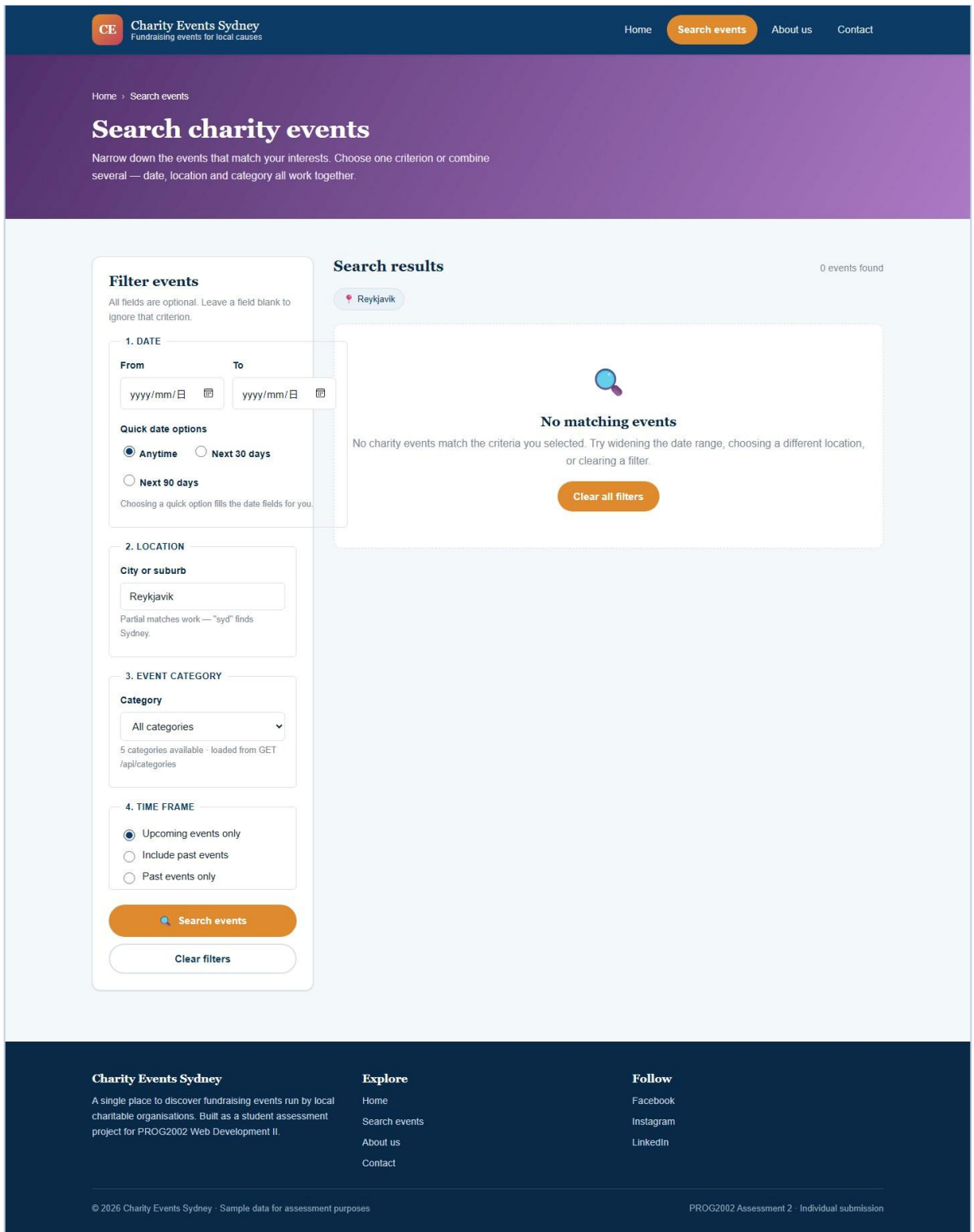


Figure 9 — The empty-result state. An empty search is treated as a normal outcome rather than an error: the page explains that nothing matched and offers to clear the filters, which is the DOM-driven recovery action.


Charity Events Sydney
Fundraising events for local causes

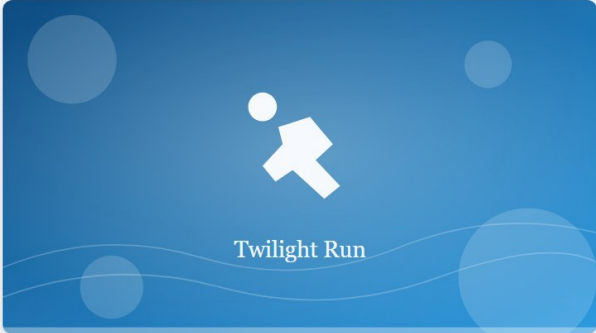
[Home](#)
[Search events](#)
[About us](#)
[Contact](#)

[Home](#) > [Search events](#) > [Harbour Lights Twilight Run 2025](#)

[FUN RUN](#)
[IN ABOUT 3 WEEKS](#)

Harbour Lights Twilight Run 2025

Sat, 17 Oct 2026
4:30 pm – 8:00 pm
Barangaroo Reserve, Barangaroo
47% of goal raised



TICKET PRICE

\$45.00

Each \$45.00 ticket is a donation towards the \$60,000 goal.

Register

Hosted by


Harbour Lights Foundation
To light the way for families facing childhood illness.

hello@harbourlights.org.au
02 5550 1200
[Organisation website](#)

[← Back to search](#)

[← Back to home](#)

About this event

A flat, scenic 5 km or 10 km course along the harbour foreshore at sunset.

Goal vs. progress

Fund 12 months of family accommodation near the children's hospital.

Raised so far

47%

\$28,450 raised

Goal: \$60,000

\$31,550 still needed to reach the goal of \$60,000.

Event details

DATE AND TIME
Sat, 17 Oct 2026, 4:30 pm – 8:00 pm

VENUE
Barangaroo Reserve 1 Sample Street, Barangaroo Sydney

CATEGORY
Fun Run

TICKETS
\$45.00 per person · capacity 1200 attendees

Charity Events Sydney
A single place to discover fundraising events run by local charitable organisations. Built as a student assessment project for PROG2002 Web Development II.

Explore
[Home](#)
[Search events](#)
[About us](#)
[Contact](#)

Follow
[Facebook](#)
[Instagram](#)
[LinkedIn](#)

© 2026 Charity Events Sydney · Sample data for assessment purposes

PROG2002 Assessment 2 · Individual submission

Figure 4 — The Event detail page for ?eventId=1. The identifier arrives in the URL query string, the page fetches that one event, and it presents the full description, the Goal vs. progress panel, the event facts and the ticket box.

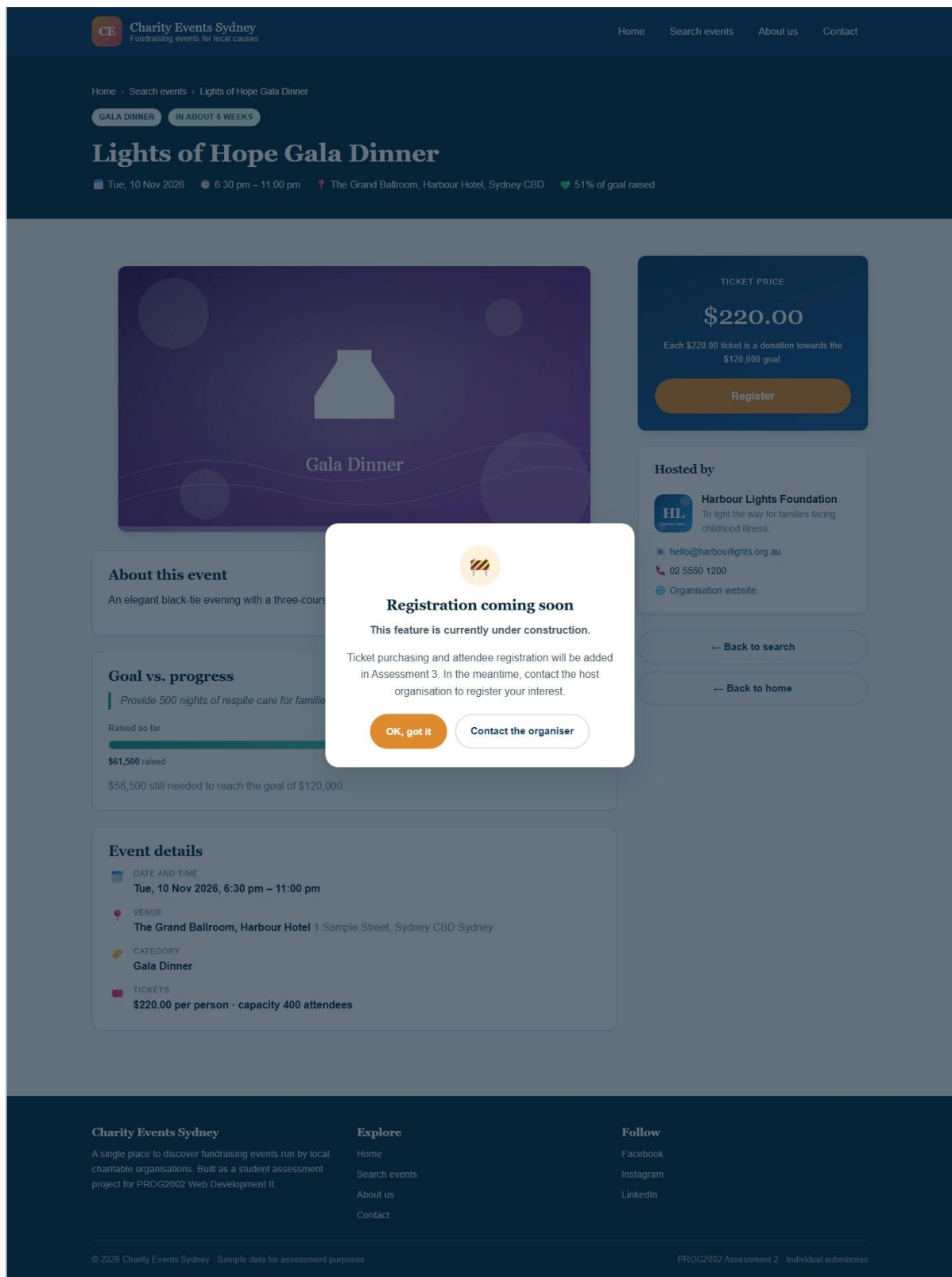


Figure 5 — The Register button opens a modal dialog reading "This feature is currently under construction." The dialog traps focus, closes on Escape or a backdrop click, and returns focus to the button.

Navigation design

- A **single, identical menu** appears in a sticky header on all three pages, so the user never has to use the browser back button to change section. The brief requires the menu on every page; making it sticky also keeps it reachable on long listings.
- The **current page is highlighted** with a filled pill and `aria-current="page"`, so the user always knows where they are in the site.
- **Breadcrumbs** appear on the Search and Event detail pages ("Home › Search events › Event name"), which gives a second, explicit way back – important because users arrive on the detail page from two different starting points.
- Every event card links to its detail page from **three places** (the title, the "View details" button and the card image area), which matches the way people actually click.
- On small screens the menu collapses behind a toggle button; the layout also switches to a single column so the filter panel and results stack sensibly on a phone.

Choosing the right input control for each criterion

The brief asks for the best input control for each data type. The choices and their reasoning are set out below.

Criterion	Data type	Control used	Why this control
Date	Range of dates (two bounds)	Two <code><input type="date"></code> pickers plus "Anytime / Next 30 days / Next 90 days" radio buttons	A date picker prevents impossible input such as "32/13/2025" and removes all format ambiguity (DD/MM vs MM/DD). The quick options cover the most common request ("what is on soon?") in a single click.
Location	Free text, semi-structured	Text field with a <code><datalist></code> of known suburbs and a "partial match" hint	A drop-down would hide events in suburbs the user did not think of, and a location list grows as organisations are added. The datalist gives suggestions without restricting input, and the API performs a partial match ("syd" finds Sydney).
Category	Small, fixed enumeration	Drop-down loaded from <code>GET /api/categories</code>	A closed list of five values is ideal for a select control: every option is valid, the user cannot mistype it, and the list stays in step with the database because it is fetched rather than hard-coded.

Criterion	Data type	Control used	Why this control
Time frame	Three mutually exclusive states	Radio buttons (Upcoming only / Include past / Past only)	Radio buttons make the mutual exclusivity visible, which prevents the contradictory request "upcoming and past" that a pair of checkboxes would allow.

Feedback, error handling and accessibility

- **Loading states.** Skeleton cards are displayed while a request is in flight, so the page does not appear broken on a slow connection and the layout does not jump when data arrives.
- **Validation before the request.** The search form checks the date range and the location text locally and marks the offending field with `aria-invalid` before calling the API, so obvious mistakes are reported instantly instead of after a round trip. Server-side validation still runs and returns HTTP 400 with a list of problems, which the page displays as well.
- **Empty states.** "No upcoming events" and "No matching events" are distinct messages with a relevant action ("Clear all filters"), because an empty result is a normal outcome of a search, not an error.
- **Error messages.** Network failures, a missing event and validation problems all produce a human-readable message built with the DOM, with a retry action where retrying makes sense.
- **Accessibility.** Semantic landmarks (header, nav, main, footer), a skip link, labelled form controls, alt text on every image, `aria-live` regions around the result lists so screen readers announce new results, focus management in the modal, and a reduced-motion media query. Colour contrast was checked against the WCAG AA threshold for body text.
- **Consistency.** One stylesheet defines the colour, spacing and type scale as CSS custom properties, so the three pages share a single visual language and a change in one place updates the whole site.

Data Schema

The database is called **charityevents_db** and was created in MySQL using InnoDB tables with the utf8mb4 character set. It contains three tables: organisations, categories and events. The design follows a normalised structure in which the two reference tables are described once and the events table stores a foreign key to each, so a category name or an organisation's contact details exist in exactly one row and cannot drift out of step.

Entity relationship overview

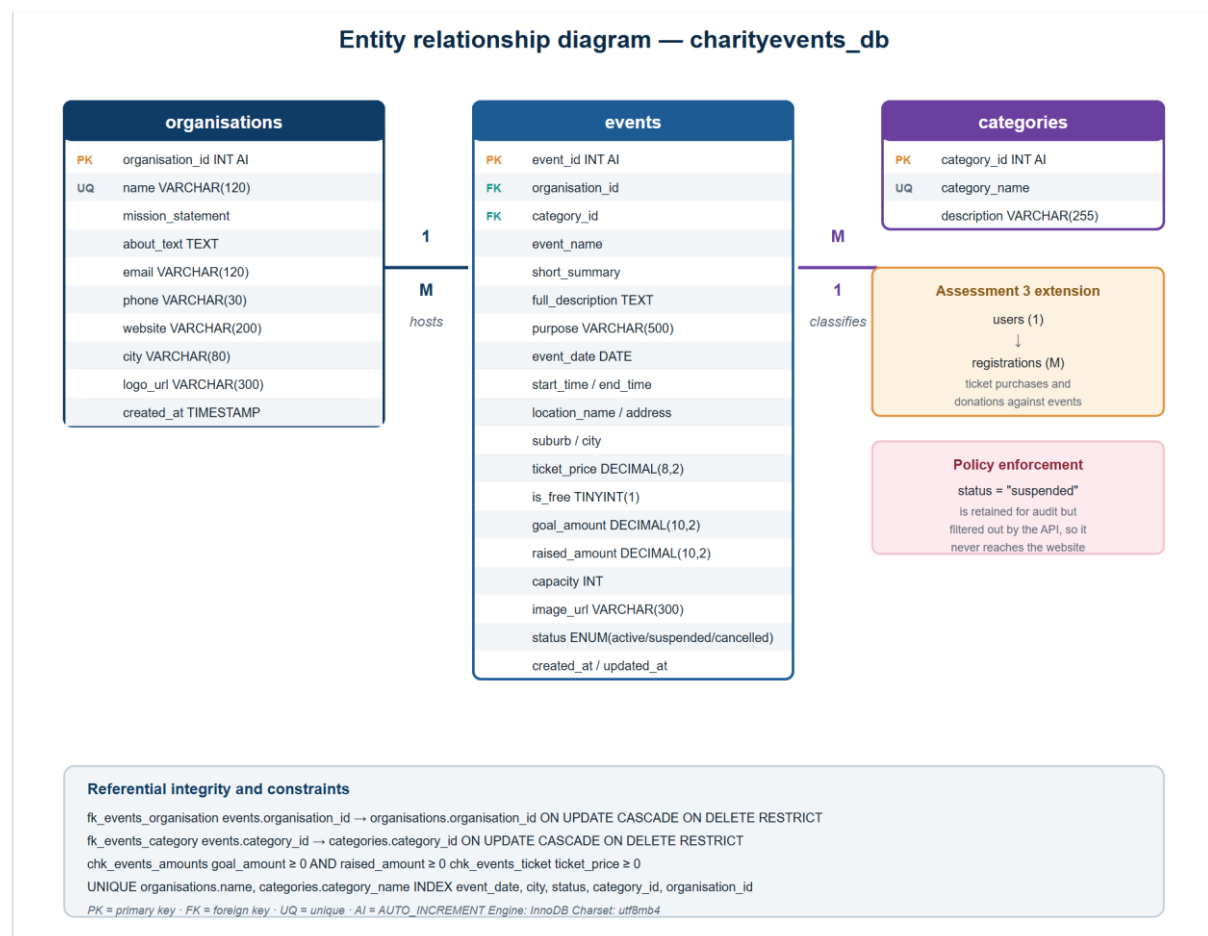


Figure 3 — Entity relationship diagram of charityevents_db: three tables, their keys, the cardinality of the two relationships, and the referential integrity rules enforced by MySQL.

Table definitions

The three tables and the role of each column group are summarised below. The complete CREATE TABLE statements are supplied in the accompanying SQL file, which the marker can import to recreate the database exactly.

Table	Key columns	Purpose
organisations	PK organisation_id; name (unique)	The charitable organisations that host events. Stores the public-facing identity and contact details shown on the Home page and on the "Hosted by" panel of the detail page.
categories	PK category_id; category_name (unique)	The five event categories (Fun Run, Gala Dinner, Silent Auction, Concert, Community Drive). Drives the category filter drop-down on the Search page and the category badge on every card.
events	PK event_id; FK organisation_id; FK category_id	The central entity. Holds description, schedule, venue, ticket pricing, the fundraising goal and amount raised, the policy status and the image used on the cards.

Relationships and integrity rules

- **organisations (1) —< events.** One organisation hosts many events. Enforced by `fk_events_organisation` on `events.organisation_id` with `ON UPDATE CASCADE` and `ON DELETE RESTRICT`. `RESTRICT` is deliberate: deleting an organisation must not silently remove its fundraising history.
- **categories (1) —< events.** One category classifies many events. Enforced by `fk_events_category` with `ON DELETE RESTRICT`, which stops a category being deleted while events still reference it.
- **Check constraints.** `chk_events_amounts` guarantees that `goal_amount` and `raised_amount` are never negative, and `chk_events_ticket` guarantees that `ticket_price` is never negative.
- **Status enumeration.** The `status` column accepts only "active", "suspended" or "cancelled", which makes it impossible to store a misspelled status that the API filter would then miss.
- **Indexes.** Indexes on `event_date`, `city`, `status`, `category_id` and `organisation_id` support the three queries the API actually runs: the upcoming feed (`status + event_date`), the search filters (`city`, `category_id`, `event_date`) and the joins to the two reference tables.
- **Timestamps.** `created_at` and `updated_at` (with `ON UPDATE CURRENT_TIMESTAMP`) provide a basic audit trail for when an event was published and last changed.

Sample data

The database is populated with realistic sample data that satisfies the brief's requirement of at least eight sample events and several categories. It contains **3 charitable organisations**, **5 categories** and **14 events**: ten active events dated in the future, three events dated in the past (so the "past" classification can be demonstrated), and one suspended event that must never appear on the website.

Event dates are stored relative to the current date when the SQL file is imported, so the upcoming events remain in the future whenever the marker imports the database and the "past" and "upcoming" states can both be demonstrated at any time.

Forward compatibility with Assessment 3

Assessment 3 adds ticket purchasing and an administration side. The schema was designed so that those features extend the existing model rather than replacing it: a users table and a registrations table (event_id, user_id, quantity, amount_paid, status) would sit between users and events, and the existing goal_amount and raised_amount columns already provide the aggregation target that a registration would update. No column in the current three tables would need to change.

API design

1. Main API endpoints

The API exposes two resources – events and categories – plus the organisations that host the events. All endpoints are read-only and return JSON. The table lists every endpoint together with the page that consumes it.

Method and URL	Purpose	Consumed by
GET /api/events	Returns active, non-suspended events. Defaults to upcoming events only; accepts status=upcoming past all and limit.	Home page (upcoming listing and the "show past events" toggle)
GET /api/events/search	Searches active events by city or suburb, category and a date range; the criteria are optional and combinable.	Search events page
GET /api/events/:id	Returns the complete record for one event, including its category, its host organisation, ticket information and fundraising progress.	Event detail page
GET /api/categories	Returns the five event categories with the number of upcoming events in each.	Search page category filter and Home page figures
GET /api/organisations	Returns the charitable organisations with their mission statements and contact details.	Home page "who we support" panel
GET /api/health	Reports whether the API is running and whether it can reach the database.	Diagnostics while marking and during the demo video

All endpoints share one response envelope, which keeps the client-side handling uniform: a successful response is { success: true, count, data } and a failed one is { success: false, message, errors }.

2. Detailed endpoint specification

The endpoint described in full below is the search endpoint, because it is the most complex of the three page-feeding endpoints: it accepts multiple optional criteria, validates them, and is the feature the demo video focuses on.

GET /api/events/search

Purpose. To return the charity events that match whatever combination of date, location and category criteria the user has chosen on the Search page. It also powers the category and location facets, so the search page is driven entirely by this

one call. Suspended events and events that have been cancelled are never returned, regardless of the criteria supplied.

Request

The endpoint takes no request body: it is a GET request, so all input arrives as **query-string parameters**. Every parameter is optional; omitting all of them returns every public event.

Parameter	Type	Required	Rules and behaviour
city	string	No	Case-insensitive partial match against the event city, suburb and venue name. Minimum two characters; letters, spaces, hyphens and apostrophes only.
categoryId	integer	No	Must be a positive integer matching a row in categories. Invalid or non-numeric values produce HTTP 400.
dateFrom	date (YYYY-MM-DD)	No	Inclusive lower bound on the event date. Must be a real calendar date; "31-12-2025" is rejected.
dateTo	date (YYYY-MM-DD)	No	Inclusive upper bound on the event date. Must not be earlier than dateFrom.
status	enum	No	One of upcoming, past or all. Defaults to all. "upcoming" returns events dated today or later, "past" returns earlier events.

Example request – a user searching for upcoming fun runs in Sydney:

```
GET /api/events/search?city=Sydney&categoryId=1&status=upcoming
```

Equivalent with a date range instead of a category:

```
GET /api/events/search?dateFrom=2025-10-01&dateTo=2025-12-31
```

Response

A successful response returns HTTP 200 with the standard envelope. The data array contains the same event shape used by every other endpoint, so the client can render results with the same card component it uses on the Home page.

```
{
  "success": true,
  "count": 2,
  "filter": {
    "city": "Sydney", "categoryId": 1,
    "dateFrom": null, "dateTo": null, "status": "upcoming"
  },
  "data": [
    {
      "eventId": 1,
      "eventName": "Harbour Lights Twilight Run 2025",
      "shortSummary": "A 5 km and 10 km sunset run along the harbour foreshore.",

```

```

    "fullDescription": "Join hundreds of runners and walkers for a flat,
    scenic ...",
    "purpose": "Fund 12 months of family accommodation near the children's
    hospital.",
    "category": { "categoryId": 1, "categoryName": "Fun Run" },
    "organisation": { "organisationId": 1, "name": "Harbour Lights Foundation",
    "missionStatement": "...", "email": "...", "phone":
    "..."} },
    "date": {
    "isoDate": "2025-10-20", "displayDate": "Mon, 20 Oct 2025",
    "startTime": "4:30 pm", "endTime": "8:00 pm"
    },
    "displayStatus": "upcoming",
    "daysUntil": 21,
    "location": { "venue": "Barangaroo Reserve", "address": "Munn Street",
    "suburb": "Barangaroo", "city": "Sydney",
    "full": "Barangaroo Reserve, Barangaroo Sydney" },
    "ticket": { "price": 45, "isFree": false, "priceLabel": "$45.00",
    "capacity": 1200 },
    "goal": { "amount": 60000, "raised": 28450, "progressPercent": 47,
    "remaining": 31550 },
    "imageUrl": "assets/images/event-twilight-run.svg",
    "status": "active"
  }
]
}

```

Note how much of the page logic the response removes. The client does not have to compare dates to decide whether an event is upcoming (`displayStatus`), calculate a countdown (`daysUntil`), format a date or price (`displayDate`, `priceLabel`) or work out a progress-bar width (`progressPercent`). All of it is computed once, on the server, from the database values.

Error responses

Invalid input never reaches MySQL. Each parameter is validated first and every problem found is collected, so the user is told about all of them at once rather than one per attempt.

Status	When it is returned	Response body
200 OK	Valid criteria, matching events found	{ success: true, count, filter, data: [...] }
200 OK	Valid criteria, no matching events	{ success: true, count: 0, filter, data: [] } – an empty result is not an error
400 Bad Request	A malformed date, a reversed date range, a non-numeric categoryId or an unknown status	{ success: false, message, errors: [...] } with one entry per problem
404 Not Found	The requested event does not exist or is suspended	{ success: false, message }
500 / 503	The database is unreachable or a query fails	{ success: false, message, detail }

3. Choice of HTTP methods

The brief asks for the reasoning behind the HTTP method chosen for each endpoint. In Assessment 2 every operation is a read, so every endpoint uses **GET**, and that is a deliberate REST decision rather than a limitation of the implementation.

Endpoint	Method used	Why this method (and not another)
/api/events	GET	Retrieving a collection changes nothing on the server, so GET is correct: it is a safe method (no side effects) and idempotent (calling it twice returns the same result). It also allows the response to be cached by the browser or a proxy and lets the user bookmark or refresh the URL safely. POST would imply that a new event is being created, which it is not.
/api/events/search	GET	The search criteria are filters over an existing collection, not a new resource, so they belong in the query string rather than in a request body. Keeping the criteria in the URL makes a filtered search shareable and bookmarkable, and preserves the safety and idempotency of GET. Using POST for a search would break the back button and prevent caching for no benefit.
/api/events/:id	GET	A single event is identified by its URL, which is the core REST principle of addressable resources. GET is again safe and idempotent, and a 404 is the natural way to report that the resource does not exist (which is also how suspended events are hidden).
/api/categories	GET	A read of a small reference collection. GET allows aggressive caching, which matters because the list is requested on every search page load.
/api/organisations	GET	A read of a collection with no side effects; GET is the only method that fits.
/api/health	GET	A diagnostic read that must be safe to call at any time, including repeatedly from a monitoring tool.

The methods that are absent are as significant as the one that is present. **POST, PUT, PATCH and DELETE are intentionally not implemented** because Assessment 2 is scoped to read-only consumption of the data; the brief explicitly states they are not required and that they will be developed in Assessment 3. The API was nevertheless structured so they can be added without redesign: a POST /api/events would create the resource that GET /api/events already lists, a PUT /api/events/:id would replace the resource that GET /api/events/:id currently returns, and a DELETE would remove it – the same URLs, the methods that REST assigns to them.

4. RESTful design and security considerations

- **Resources, not actions.** URLs are nouns (/api/events) and the HTTP method expresses the verb. There are no URLs such as /getEvents or /searchEvents.
- **Logical hierarchy.** The collection lives at /api/events and an individual member at /api/events/:id. The search is expressed as a filtered view of the collection through query parameters rather than as a separate, verb-like endpoint.
- **SQL injection protection.** Every value reaches MySQL as a bound parameter in a prepared statement (the ? placeholders used throughout routes/events.js); user text is never concatenated into SQL. Ordering clauses are selected from a fixed internal whitelist, and the limit value is bound as a parameter and clamped to a maximum of 100.
- **Information disclosure.** The API returns the same event shape everywhere and hides suspended events behind a 404, so it does not reveal that a withheld event exists. Error responses carry a human-readable message and, where useful, an error code, but never a stack trace or a raw database error.
- **Credential handling.** Database credentials are read from environment variables through a .env file that is excluded from version control by .gitignore; only a .env.example template is committed.
- **Correct status codes.** 200 for success, 400 for invalid input, 404 for a missing resource, and 500/503 for server or database failures, so the client can distinguish a user error from an infrastructure problem.
- **CORS.** The API enables CORS so the client-side website can consume it even when the two are served from different origins, which is how the two should be submitted as separate ZIP files.

4. Implementation: how the three layers connect

The sequence below is the actual code path for a search request. It is included because the markers' criteria reward an understanding of how the client and the server cooperate, not only of what each file contains in isolation.

Step 1 — the browser sends the request

The search page collects the form values into a criteria object and passes it to a single fetch wrapper. Because the wrapper returns a Promise, the page can use `async/await` and keep the error handling in one `try/catch` block:

```
// client/js/search.js
const response = await API.searchEvents({
  city:      criteria.city,
  categoryId: criteria.categoryId,
  dateFrom:  criteria.dateFrom,
  dateTo:    criteria.dateTo,
  status:    criteria.status
});

// client/js/api.js - one shared fetch wrapper for every endpoint
async function request(path, options = {}) {
  const response = await fetch(`${BASE_URL}${path}`, {
    headers: { Accept: 'application/json' }, ...options
  });
  const body = await response.json();
  if (!response.ok) {
    throw new ApiError(body.message, response.status, body.errors);
  }
  return body;
}

searchEvents: (criteria = {}) =>
  request(`/api/events/search${buildQueryString(criteria)}`)
```

`buildQueryString()` drops empty values, so an untouched filter field never reaches the server as `"&city="` and the API only applies the criteria the user actually chose. That is what makes "one or several criteria" work without any special-casing in the route.

Step 2 — Express validates and queries

The route accumulates its WHERE conditions and their bound parameters together, so a criterion can never be added without its placeholder. Every value is bound; only the ORDER BY clause is built from internal fixed strings:

```
// api/routes/events.js
const conditions = ['e.status IN (?, ?)']; // suspended is never public
const params = [...PUBLIC_STATUSES];

if (city && String(city).trim() !== '') {
  conditions.push('(e.city LIKE ? OR e.suburb LIKE ? OR e.location_name LIKE ?)');
  params.push(`%${term}%`, `%${term}%`, `%${term}%`);
}
if (categoryId !== undefined && String(categoryId).trim() !== '') {
  conditions.push('e.category_id = ?');
  params.push(parsedCategory);
}
if (dateFrom) { conditions.push('e.event_date >= ?'); params.push(validFrom); }
if (dateTo)   { conditions.push('e.event_date <= ?'); params.push(validTo); }
```

```
const rows = await query(`${BASE_SELECT} WHERE ${conditions.join(' AND ')} ORDER BY ...`, params);
```

Step 3 — the row becomes the JSON contract

eventMapper.js is the boundary between the database and the browser. It is the reason the client never has to compare dates or calculate percentages:

```
// api/utils/eventMapper.js
return {
  eventId: row.event_id,
  eventName: row.event_name,
  category: { categoryId: row.category_id, categoryName: row.category_name },
  date: { isoDate: toDateString(row.event_date),
    displayDate: formatDate(row.event_date),
    startTime: formatTime(row.start_time),
    endTime: formatTime(row.end_time) },
  displayStatus: classifyEvent(row.event_date), // 'upcoming' | 'past'
  daysUntil: daysUntil(row.event_date),
  ticket: { price, isFree, priceLabel, capacity },
  goal: { amount, raised, progressPercent, remaining }
};
```

Step 4 — the browser renders it into the DOM

The resolved Promise is turned into markup by one shared function, which both the Home page and the Search page use. All API text is escaped before it is written to innerHTML:

```
// client/js/ui.js
function insertEventCards(container, events, options = {}) {
  if (!events.length) { showEmpty(container, options.emptyMessage); return; }
  container.innerHTML = events
    .map((event) => renderEventCard(event, options))
    .join('');
}

// client/js/home.js
const response = await API.getEvents({ status: 'upcoming' });
insertEventCards(eventsContainer, response.data);
```

Wiring the whole path together: a user types "Sydney" in the location field, selects "Fun Run" and presses Search. The form calls preventDefault(), validates the criteria with the DOM, updates the URL with history.replaceState() so the search can be bookmarked, awaits the fetch, and renders the three matching cards that Figure 8 shows.

5. Testing

The API is covered by an automated test suite (api/test/api_test.js) that exercises every endpoint and its error cases, and an end-to-end smoke test (api/test/smoke_test.js) that boots the real Express server and checks the website and the API together. The suite verifies, among other things, that the Home feed

returns only upcoming events, that at least eight events are returned, that the suspended event is absent from every public response and returns 404 on direct access, that each of the three search criteria filters correctly and combines with the others, and that malformed input produces HTTP 400 rather than a server error. During development the endpoints were also exercised in Postman and in the browser, and those requests are demonstrated in the demo video.

Generative AI Use Declaration

Choose the statement that applies and delete the other one before submitting.

Statement A – if GenAI tools were used:

I acknowledge that I have used GenAI tools to complete this assessment. I used _____ (GenAI tool(s)) to _____ (specific purpose(s) of using GenAI) within the parameters outlined in the Assessment Brief and by the Unit Assessor.

Statement B – if GenAI tools were not used:

I acknowledge that I have not knowingly used GenAI to complete this assessment.

Academic Integrity Declaration

By submitting this assessment, I declare that I have read and understood SCU's Academic Integrity policies and referencing guidelines. I am aware of the consequences of academic misconduct and confirm that this submission is my own original work, referenced appropriately, and has not been previously submitted. I authorise its reproduction for authentication purposes and understand the implications of a false declaration. I have adhered to guidelines regarding Generative AI.

Submission checklist

Deliverable	Detail	Ready
Project documentation	This report, using the supplied template	☐
API source code	usernameA2-api.zip (Node.js, ExpressJS, MySQL, event_db.js, routes, utils)	☐
Client-side source code	usernameA2-clientside.zip (HTML, CSS, JavaScript, DOM)	☐
Database SQL file	database/charityevents_db.sql, importable in MySQL Workbench	☐
GitHub repository link	Private repository with incremental commits and descriptive messages; marker invited as collaborator	☐
Demo video	Maximum 15 minutes, uploaded to SCU OneDrive with a shareable link	☐
GenAI declaration	Statement A or B above, with the other removed	☐

Appendix A — Repository structure

The two source ZIP files are submitted separately as required. The table maps the assessment parts to the files that implement them, so the marker can navigate the submission quickly.

Submission file	Path inside the ZIP	What it contains
usernameA2-api.zip	api/server.js	Express application: middleware, route mounting, static hosting, error handler
usernameA2-api.zip	api/event_db.js	The required MySQL connection file (mysql2 connection pool)
usernameA2-api.zip	api/routes/events.js	GET /api/events, /api/events/search, /api/events/:id
usernameA2-api.zip	api/routes/categories.js	GET /api/categories
usernameA2-api.zip	api/routes/organisations.js	GET /api/organisations
usernameA2-api.zip	api/utils/dateUtils.js	Upcoming/past classification, countdowns, formatting
usernameA2-api.zip	api/utils/eventMapper.js	Database row → JSON contract, progress calculation
usernameA2-api.zip	api/assets/images/	19 SVG assets: event artwork, organisation logos, hero banners
usernameA2-api.zip	api/database/charityevents_db.sql	Importable database: schema plus all sample data
usernameA2-api.zip	api/test/api_test.js	Automated endpoint test suite (47 checks)
usernameA2-api.zip	api/test/smoke_test.js	End-to-end test of the real server (25 checks)
usernameA2-api.zip	api/.env.example	Template for the database credentials (.env is git-ignored)
usernameA2-clientside.zip	index.html / search.html / event.html	The three required pages
usernameA2-clientside.zip	css/styles.css	Single stylesheet: design tokens, components, responsive rules
usernameA2-clientside.zip	js/api.js	Fetch wrapper and Promise handling for every endpoint
usernameA2-clientside.zip	js/ui.js	DOM rendering: cards, messages, empty states, modal, navigation
usernameA2-clientside.zip	js/home.js / search.js / event.js	Page logic for each of the three pages

Submission file	Path inside the ZIP	What it contains
usernameA2-clientside.zip	js/config.js	Hard-coded organisation text (the static Home page content)

Appendix B — Automated test evidence

Two suites ship with the project and can be re-run by the marker with a single command each. The endpoint suite uses an in-memory stand-in for MySQL so it runs without a database server; the smoke test boots the real server.js and checks the website and the API together.

```
$ cd api
$ node test/api_test.js          # 47 checks
$ node test/smoke_test.js        # 25 checks

=== PROG2002 A2 - Charity Events API test suite ===

GET /api/events (Home page feed)
PASS responds 200
PASS returns only upcoming events
PASS returns at least 8 events (brief requirement: min 8)
PASS excludes the suspended event
PASS includes the category name on every event
PASS includes goal progress between 0 and 100
PASS is sorted by date ascending

GET /api/events/search (the three required criteria)
PASS filters by location
PASS filters by category
PASS filters by date range
PASS combines several criteria at once
PASS returns an empty result set (not an error) when nothing matches

GET /api/events/search (validation)
PASS rejects a malformed date with 400
PASS rejects a reversed date range with 400
PASS rejects a non-numeric category with 400

GET /api/events/:id (Event detail page)
PASS responds 200 for a real event
PASS hides a suspended event with 404
PASS responds 400 for a non-numeric id

RESULT: 47 passed, 0 failed

=== PROG2002 A2 - end-to-end smoke test ===
PASS Home page          PASS GET /api/events
PASS Search page        PASS GET /api/events/search
PASS Event detail page  PASS GET /api/events/:id
PASS Stylesheet and scripts PASS GET /api/categories
PASS Event and logo SVGs PASS GET /api/organisations
PASS Suspended event hidden PASS Invalid input rejected

RESULT: all 25 smoke checks passed
```

Appendix C — Running the project

The complete instructions are in the README files inside each ZIP. In summary:

- **Create the database.** Open MySQL Workbench, open database/charityevents_db.sql and execute it. This creates charityevents_db with its three tables and all sample data.
- **Configure the credentials.** Copy api/.env.example to api/.env and set DB_PASSWORD to your own MySQL password. The .env file is excluded from version control.
- **Install and start.** Run "npm install" then "node server.js" in the api folder. The terminal confirms the database connection and prints the URLs.
- **Open the website.** <http://localhost:3000/index.html> for the Home page, [/search.html](http://localhost:3000/search.html) for the Search page, and [/event.html?eventId=3](http://localhost:3000/event.html?eventId=3) for an event. The API is served from the same process, so a single command runs the whole project.
- **Verify the environment (optional).** "node scripts/test_connection.js" confirms MySQL connectivity on its own, and the two test suites in Appendix B exercise every endpoint.